Bahria University, Islamabad

Department of Software Engineering

Web Engineering Lab (Fall-2025)

Instructor: Engr. Subas Bilal

Student: Waleed Ahmed

Enrollment: 01-131232-093

Lab Journal: 10

Date: 4/22/2026

Open Ended Lab:

Lab Task: Integrate Frontend and Backend for Complete Student Feedback Model

Full-Stack Integration Lab

Lab Objectives

By the end of this lab, students will be able to:

- Build a complete product management application using the MERN stack
- Connect a React frontend to a Node.js/Express backend via Axios
- Perform all CRUD operations: Create, Read, Update, and Delete
- Use React state and hooks to manage UI data dynamically
- Handle API responses and display real-time feedback to users
- Store and retrieve product data from MongoDB Atlas

Software / Tools Required

- Node.js (installed)
- Visual Studio Code
- MongoDB Atlas account
- Postman (for backend testing)
- A modern web browser (Chrome / Edge)

Lab Description

In this lab, you will integrate a React frontend with the Node.js/Express backend you built in a previous lab. The goal is to create a fully functional Product Management application that supports Create, Read, Update, and Delete (CRUD) operations. User interactions on the frontend will trigger HTTP requests to the backend API, which will read from and write to MongoDB Atlas.

References: https://www.youtube.com/watch?v=O3BUHwfHf84 | https://www.youtube.com/watch?v=98BzS5Oz5E4

Step 1: Project Structure Setup

Your MERN project folder should have the following structure:

```
NEW MERN
├── backend
│   ├── .env
│   └── server.js
└── frontend
    ├── src
    │   ├── components
    │   │   ├── App.css
    │   │   └── index.css
    │   │   └── App.jsx
    │   │   └── main.jsx
    │   └── index.html
    └── package.json
```

If you do not already have a frontend folder, create it now:

```
cd NEW MERN npm create vite@latest frontend -- --
template react cd frontend npm install npm install
axios
```

Step 2: Backend – Feedback Model (Server.js)

Create a new file Product.js inside the backend/models folder. This defines the schema for a product stored in MongoDB.

```
import dns from 'node:dns';
dns.setServers(['8.8.8.8', '8.8.4.4']);
import express from 'express';
import mongoose from 'mongoose';
import cors from 'cors';
import bodyParser from 'body-parser';
import dotenv from 'dotenv';
dotenv.config();

const app = express();
app.use(cors());
```

```
app.use(bodyParser.json());

mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true
}).then(() => {
  console.log('Connected to MongoDB');
}).catch(err => {
  console.error('MongoDB connection error:', err);
});

const feedbackSchema = new mongoose.Schema({
  studentName: {
    type: String,
    required: true
  },
  subject: {
    type: String,
    required: true
  },
  rating: {
    type: Number,
    required: true,
    min: 1,
    max: 5
  },
  comments: {
    type: String
  }
});

const Feedback = mongoose.model('Feedback', feedbackSchema);

app.post('/feedback', async (req, res) => {
  try {
    const feedback = new Feedback(req.body);
    await feedback.save();
    res.status(201).send(feedback);
  } catch (error) {
    res.status(400).send(error);
  }
});
```

```
app.get('/feedbacks', async (req, res) => {
  try {
    const feedbacks = await Feedback.find();
    res.send(feedbacks);
  } catch (error) {
    res.status(500).send(error);
  }
});

app.get('/feedbacks/:subject', async (req, res) => {
  try {
    const feedbacks = await Feedback.find({ subject: req.params.subject });
    res.send(feedbacks);
  } catch (error) {
    res.status(500).send(error);
  }
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 3: Backend – env
Ans. Done

Step 4: Frontend – App.jsx Component

Create the file frontend/src/components/ProductForm.jsx. This form allows users to add new products or update existing ones.

```
import { useState, useEffect } from 'react';
import FeedbackForm from './components/FeedbackForm';
import FeedbackList from './components/FeedbackList';
import './App.css';

function App() {
  const [feedbacks, setFeedbacks] = useState([]);
  const [selectedSubject, setSelectedSubject] = useState('');

  useEffect(() => {
    fetchFeedbacks();
```

```
  }, []);

  const fetchFeedbacks = async () => {
    try {
      const response = await fetch('http://localhost:5000/feedbacks');
      const data = await response.json();
      setFeedbacks(data);
    } catch (error) {
      console.error('Error fetching feedbacks:', error);
    }
  };

  const handleSubjectChange = (subject) => {
    setSelectedSubject(subject);
  };

  const filteredFeedbacks = selectedSubject ? feedbacks.filter(f => f.subject ===
selectedSubject) : feedbacks;

  const averageRating = filteredFeedbacks.length > 0 ? (filteredFeedbacks.reduce((sum,
f) => sum + f.rating, 0) / filteredFeedbacks.length).toFixed(1) : 0;

  return (
    <div className="App">
      <h1>Student Feedback Management System</h1>
      <FeedbackForm onSubmit={fetchFeedbacks} />
      <div>
        <label>Filter by Subject: </label>
        <select value={selectedSubject} onChange={(e) =>
handleSubjectChange(e.target.value)}>
          <option value="">All</option>
          {[...new Set(feedbacks.map(f => f.subject))].map(subject => (
            <option key={subject} value={subject}>{subject}</option>
          ))}
        </select>
        {selectedSubject && <p>Average Rating for {selectedSubject}:
{averageRating}</p>}
      </div>
      <FeedbackList feedbacks={filteredFeedbacks} />
    </div>
  );
}
```

```
export default App;
```

Step 5: Frontend – main.jsx Component

Create the file frontend/src/components/ProductList.jsx. This component fetches and displays all products, and provides Edit and Delete buttons for each.

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './index.css'

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

Step 6: Running the Full Application

You need two terminals running simultaneously – one for the backend and one for the frontend.

Terminal 1 – Start Backend

```
cd NEW MERN/backend npm
run dev
```

Expected output:

```
MongoDB connected
Server running on port 5000
```

Terminal 2 – Start Frontend

```
cd NEW MERN/frontend npm
run dev
```

Expected output:

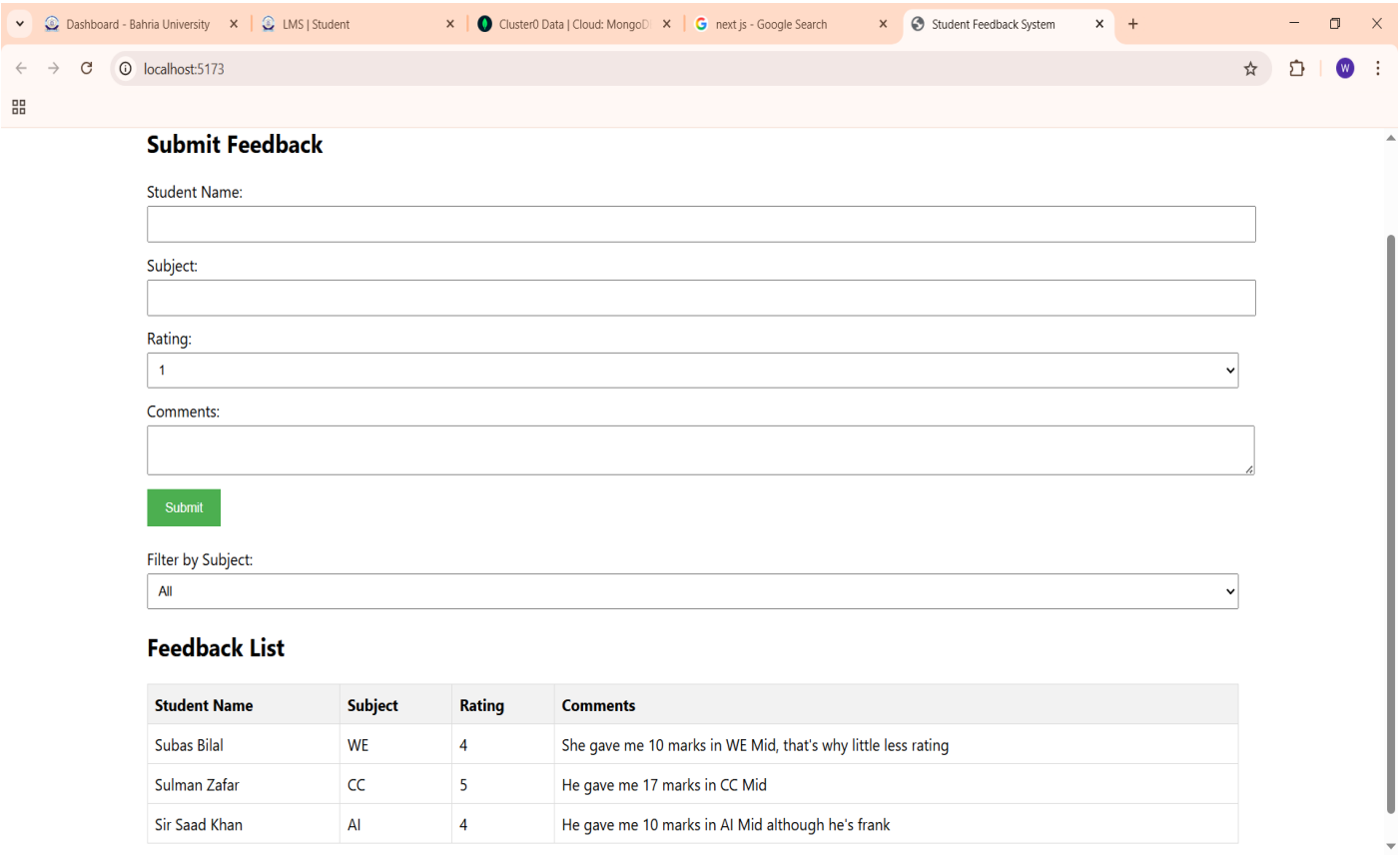
```
  VITE v5.x.x  ready in xxx ms

  ➜  Local:   http://localhost:5173/
```

Open http://localhost:5173 in your browser. You should see the Product Manager interface.

Step 7: Testing Feedback in the Browser

Perform all four CRUD operations through the browser UI and verify they work end-to-end.

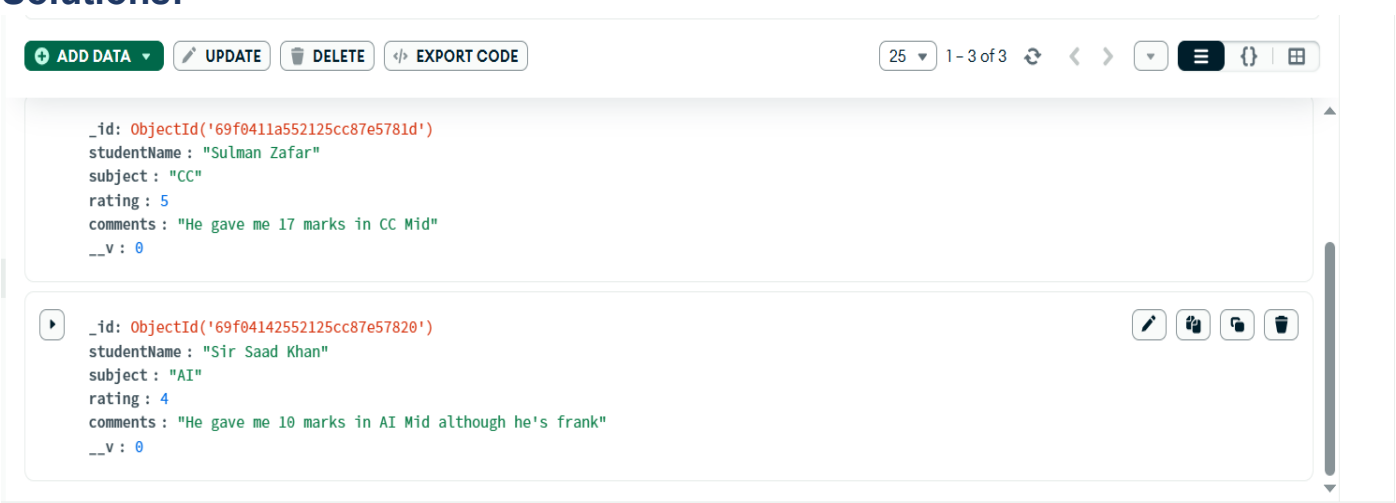
Solutions:



Step 8: MongoDB Atlas Verification

1. Open your MongoDB Atlas account.
2. Navigate to your cluster and click Browse Collections. 3. Select your database and find the products collection.
4. Confirm that the documents match the products shown in the browser.

Solutions:



Lab Submission

- Screenshot of the browser showing feedback model (at least 3 feedbacks)
- Screenshot of the MongoDB Atlas products collection
- Zip file of the complete project (both backend and frontend folders)
- All screenshots compiled into a single PDF